Tutorial

# Accelerated Real-Time C++ Audio: DSP Offloading and GPU Neural Inference

*Victor Zappi*
*College of Arts Media and Design, Northeastern University*

DAFx 26

September 1, 2026

LVX VERITAS VIRTVS
Northeastern University

# In a Nutshell

0. Intro/context/tools

1. CPU Audio

2. DSP Offloading

3. GPU Neural Audio Inference

**BENCHMARKING INTEGRATED GPU ACCELERATION OF REAL-TIME NEURAL AUDIO INFERENCE ON SNAPDRAGON**

*Avery Huang, Gautham Srinivasan*
Northeastern University
Boston, USA

*Akito van Troyer*
Berklee College of Music
Boston, USA

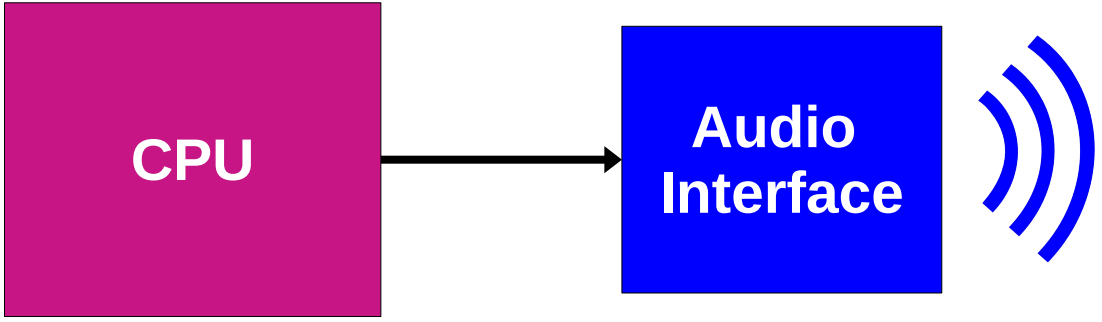
*Victor Zappi*
Northeastern University
Boston, USA

***Paper Session 9, Friday 3pm***

# 0. Intro/context/tools

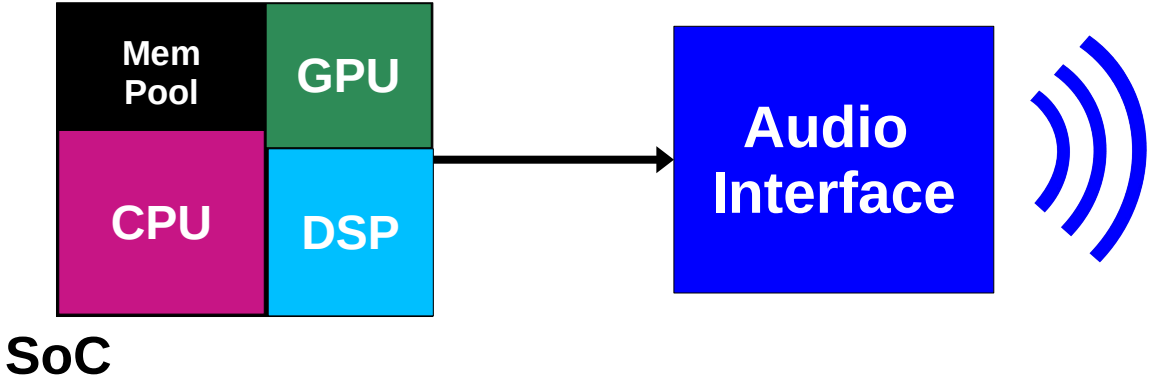
# Standard Audio Workflow

- Typically, on general purpose computers the CPU is the processor where audio samples are synthesized/processed
- They are then sent to a *separate* device that turns the digital stream into an analog signal/sound (e.g., audio interface)



# General Vs. Dedicated

- The CPU is designed to carry out all sorts of calculations, not just audio
- Audio doesn't have to run there though!
  - Nowadays, computers are often powered by **Systems on Chip** (SoCs)
  - SoCs pack *dedicated* processors alongside the general purpose CPU



# Offloading Trade-off

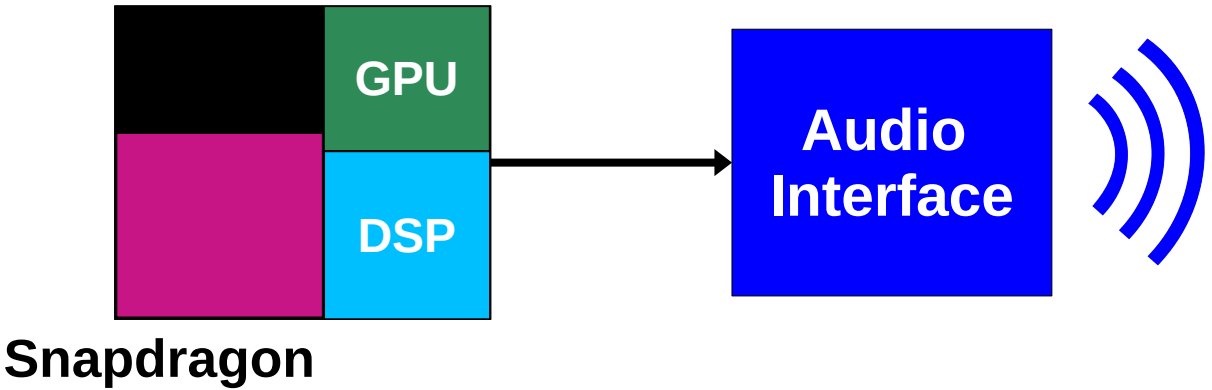
- Offloading audio to dedicated processors can pay off in several ways:
  - Performance improvement
  - Algorithmic feasibility
  - CPU headroom

  BUT
- It comes with some drawbacks:
  - Hardware and software dependent
  - Potentially complex setup
  - Constrains apply

# Audio Acceleration on Snapdragon

- Today, we'll see how to use the dedicated processors of Qualcomm's Snapdragon family of SoCs as *audio accelerators*
- We'll cover:
  - audio offloading to the Hexagon DSP
  - neural audio inference on the Adreno GPU

*Driven by custom C++ applications running from CPU!*



# Why Snapdragon?

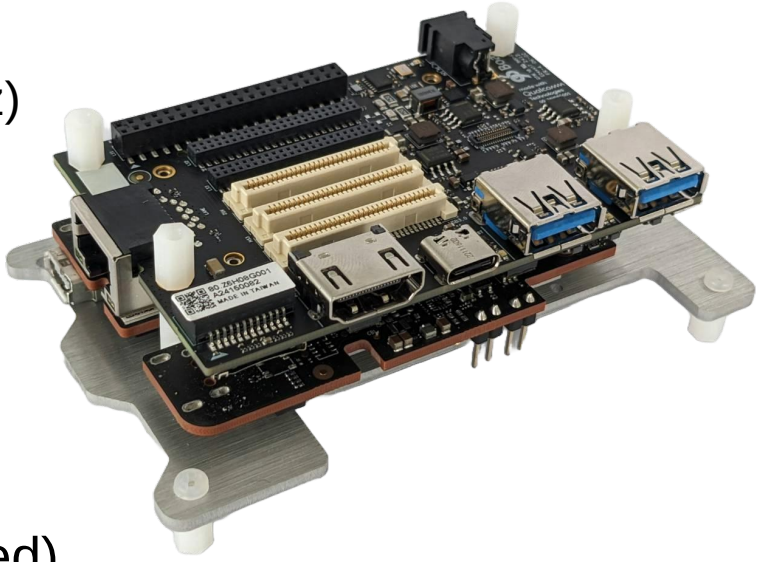
- Snapdragons are found in a variety of computing devices, from general purpose computers to dedicated devices, including:
  - Laptops
  - Android Phones
  - Arduinos
  - Audio gears


- Qualcomm provides a convenient and efficient software ecosystem (frameworks, libraries, APIs) for running code on the dedicated processors

# Board

- We’ll use the RB3 Gen 2 board, a Snapdragon IoT development board
- The SoC – QCS6490 (Kodiak)
  - CPU: Kryo 670, 8 Arm Cortex cores (1.9–2.7 GHz)
  - DSP: Hexagon V68, 1980 MPPS
  - GPU: Adreno 643 @ 812 MHz

*All recent Snapdragon SoCs are equipped with Hexagon+Adreno, so workflows are not specific to the RB3!*

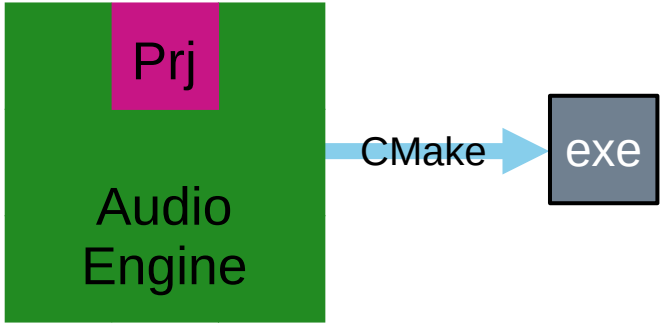


- Running Qualcomm Linux (Yocto/OpenEmbedded)
  - version 1.8, non-preemptive kernel

# 1. CPU Audio (on Snapdragon)

# Audio Engine and Projects

- We'll build and run **audio applications in C++**, using an open-source **audio engine**:
  - *github.com/victorzappi/ar-audioengine*
- Each application is a **separate project** that gets built within the engine and runs on the board
- All the projects for this tutorial can be found in this repo:
  - *github.com/victorzappi/dafx26-projects*



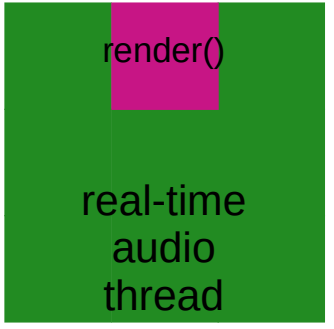
# A Minimal Project

- Each project is composed of at least a *render.cpp* file, implementing a simple API:
  - *setup()*
  - *render()* ---> This is where samples are synthesized/processed and passed down the pipeline
  - *cleanup()*
- Let’s inspect ***dafx26_projects/file_player***…

# Audio Engine’s Core

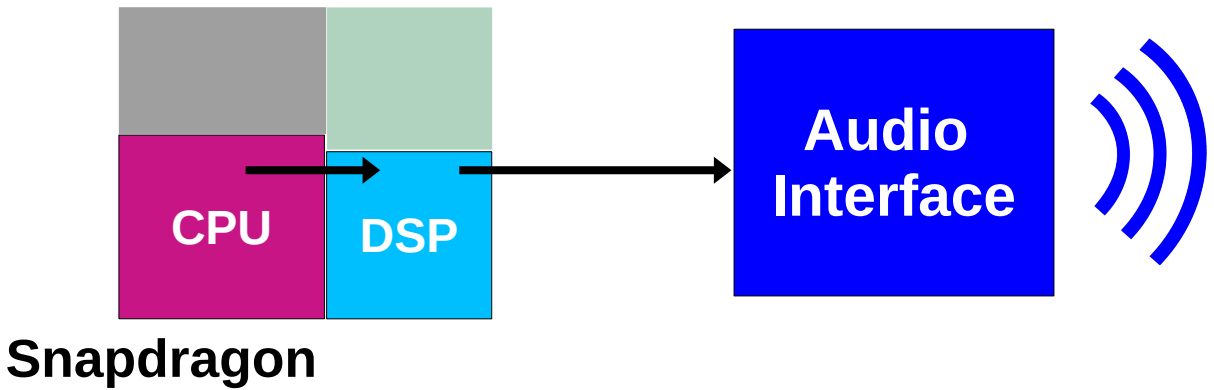
- At heart, the engine is **ordinary Linux audio code**
  - based on TinyALSA (lightweight ALSA userspace, standard on Android/embedded)
  - opens a PCM device (audio interface abstraction)
  - configures period size and period count (audio buffer), sets format and sample rate
  - real-time audio thread runs at highest priority, calls the project’s *render()* function and pushes the samples to the PCM

  HOWEVER
- This is not enough for audio to work on Snapdragon!



# DSP and Tunneling

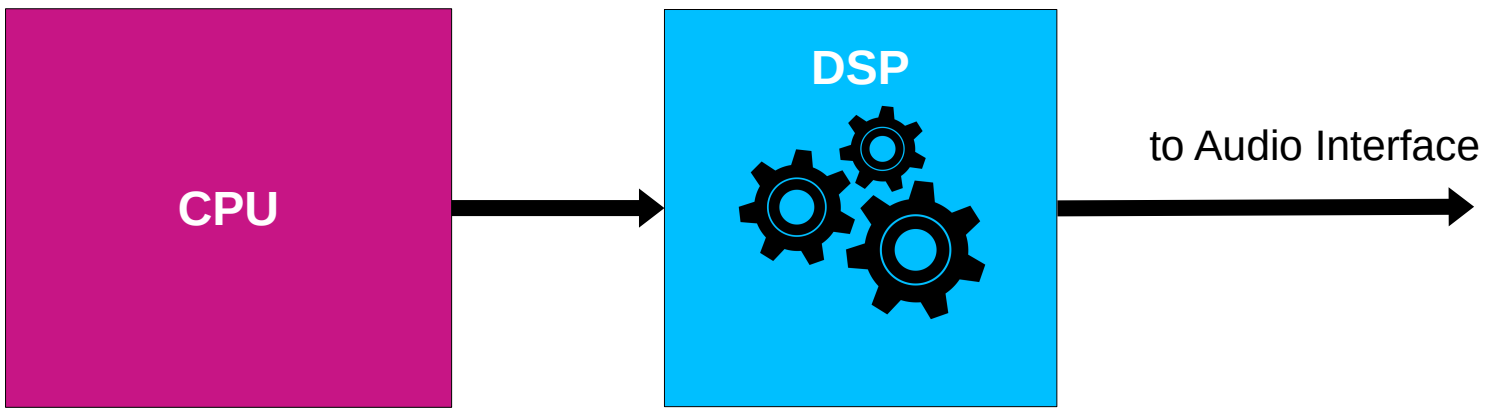
- On Snapdragon, the CPU cannot stream audio to the audio interface directly
- The stream **always passes through the DSP**
  - This is called *tunneling*



# DSP Programming

- The DSP must be programmed to decide what happens to the samples as they travel through the DSP
  - This can include doing nothing, i.e., an actual pass-through!
- No DSP programming = cannot hear anything...

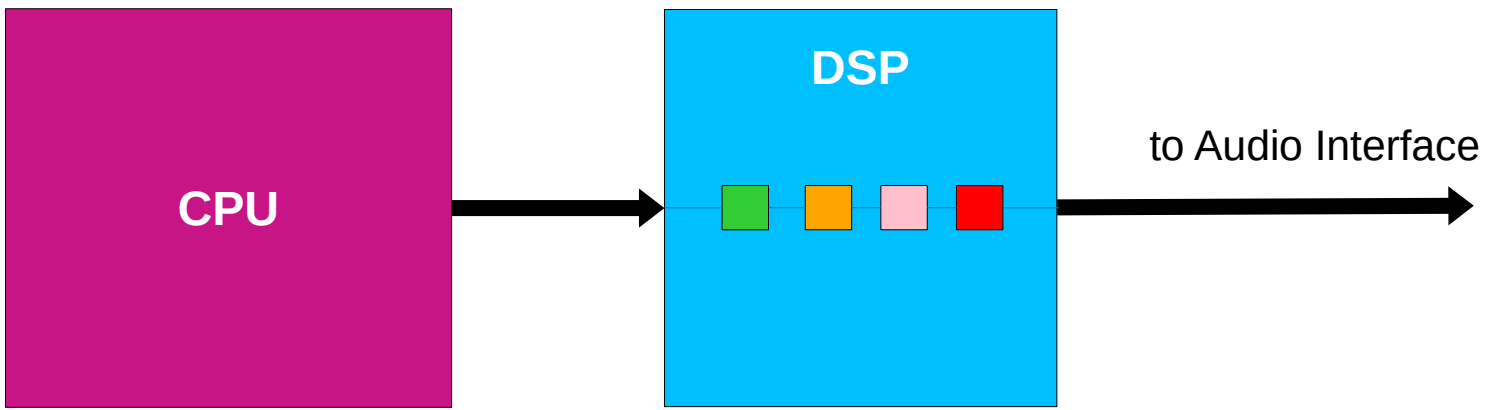
*But how do we program the DSP?*



# AudioReach

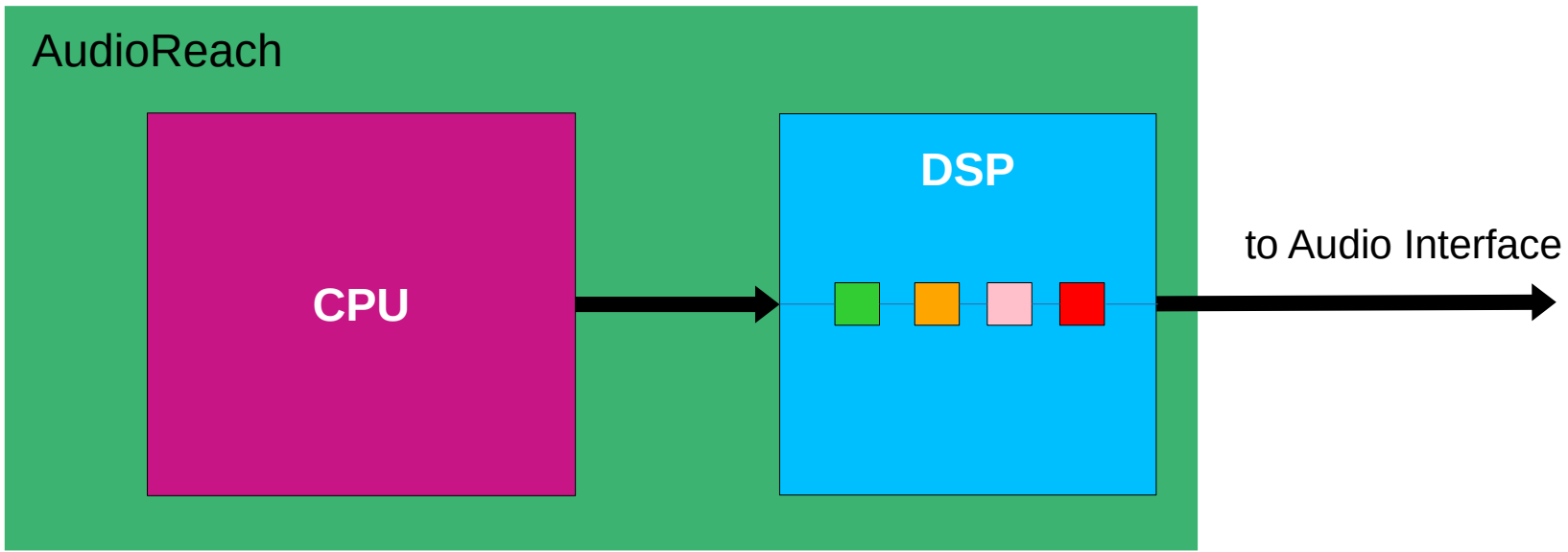
- In 2025 Qualcomm released *AudioReach*, a framework for programming the DSP via **graphs of processing modules**

*AudioReach is* ***open-source!***



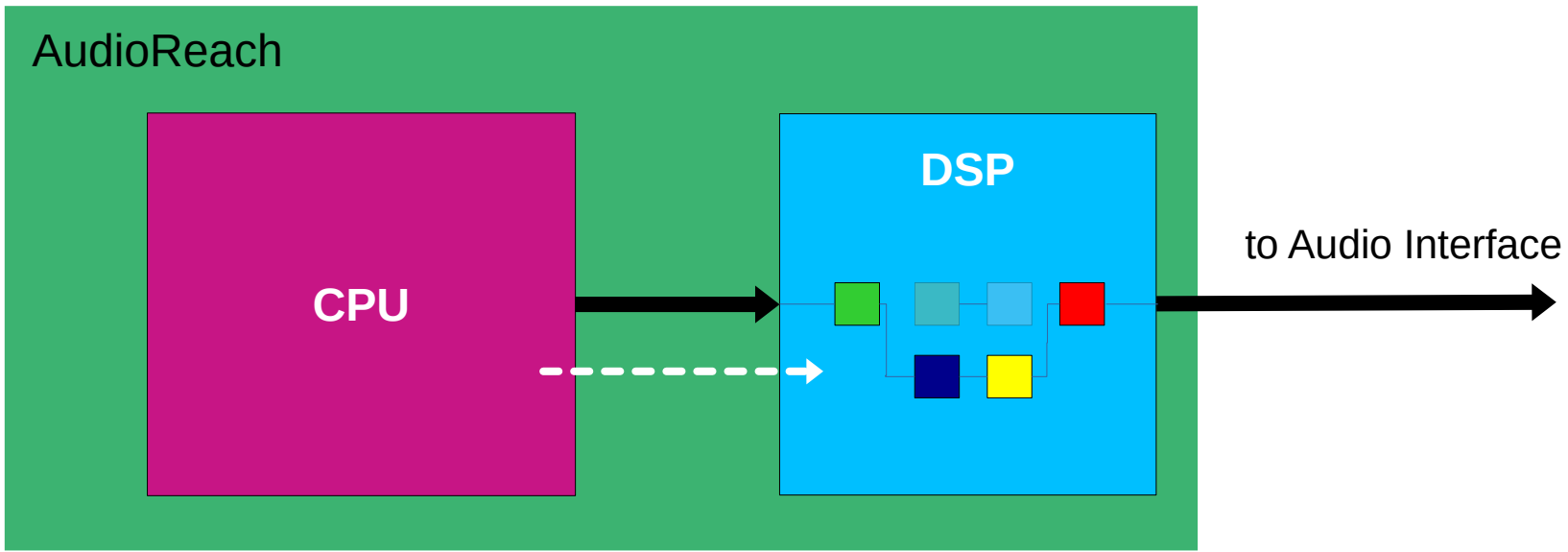
# AudioReach

- In 2025 Qualcomm released *AudioReach*, a framework for programming the DSP via **graphs of processing modules**
- It also stretches to the CPU, allowing user code to configure the DSP at run-time



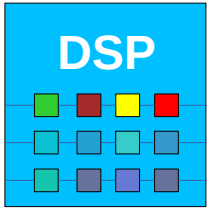
# AudioReach

- In 2025 Qualcomm released *AudioReach*, a framework for programming the DSP via **graphs of processing modules**
- It also stretches to the CPU, allowing user code to configure the DSP at run-time
  - Our C++ code must tell AudioReach to load on the DSP the graph we want to use!



# AudioReach DSP Programming

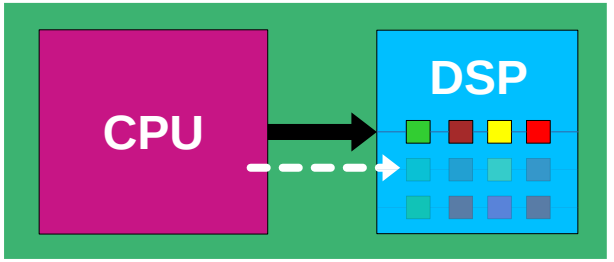
- The full AudioReach workflow is composed of two steps:
  - design a new graph (offline)




*All graphs are stored on device!*

# AudioReach DSP Programming

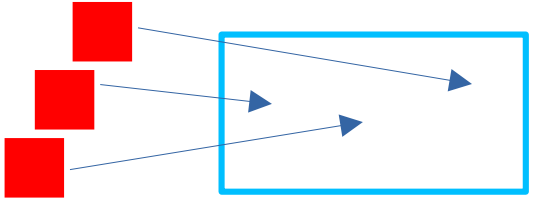
- The full AudioReach workflow is composed of two steps:
  - design a new graph (offline)
  - load that graph (run-time)




*Let's see some details of this workflow!*

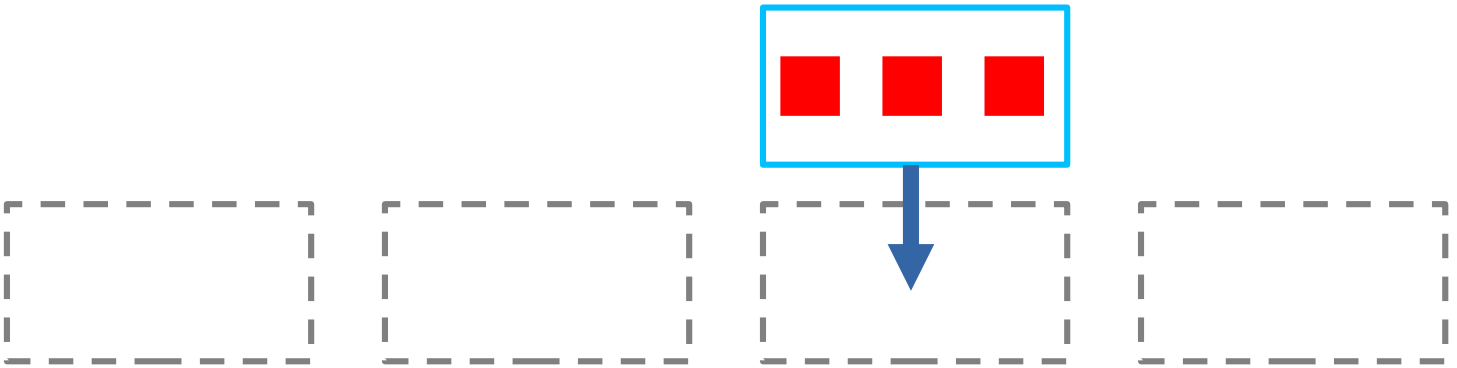
# Graph Design (Simplified)

- AudioReach comes with *AudioReachCreator*, a graphical editor to create and edit graphs and including a library of modules
- To build a graph, first modules are grouped into **sub-graphs** (SGs)



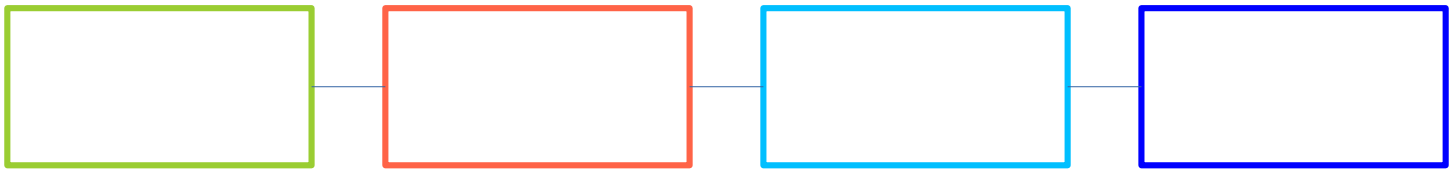
# Graph Design (Simplified)

- AudioReach comes with *AudioReachCreator*, a graphical editor to create and edit graphs and including a library of modules
- To build a graph, first modules are grouped into **sub-graphs** (SGs)
- Then each sub-graph is placed in one of four possible slots within the graph



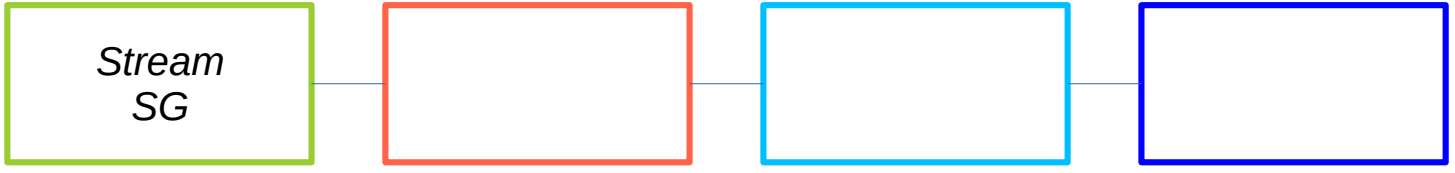
# Graph Design (Simplified)

- AudioReach comes with *AudioReachCreator*, a graphical editor to create and edit graphs and including a library of modules
- To build a graph, first modules are grouped into **sub-graphs** (SGs)
- Then each sub-graph is placed in one of four possible slots within the graph
- Slots are chosen on the basis of what the sub-graphs' modules do:



# Graph Design (Simplified)

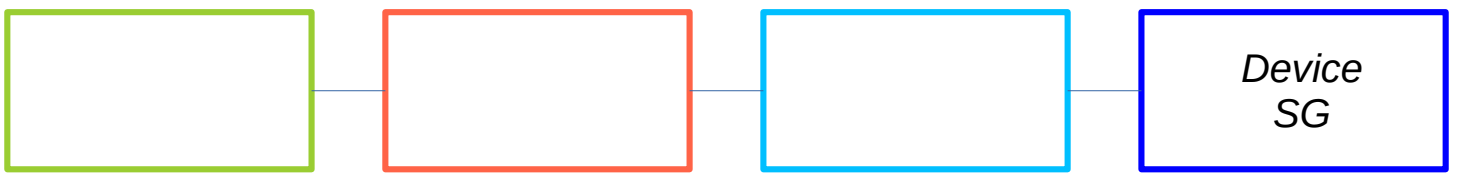
- AudioReach comes with *AudioReachCreator*, a graphical editor to create and edit graphs and including a library of modules
- To build a graph, first modules are grouped into **sub-graphs** (SGs)
- Then each sub-graph is placed in one of four possible slots within the graph
- Slots are chosen on the basis of what the sub-graphs' modules do:




*Modules that:*
- *receive samples from CPU*
- *control volume*

# Graph Design (Simplified)

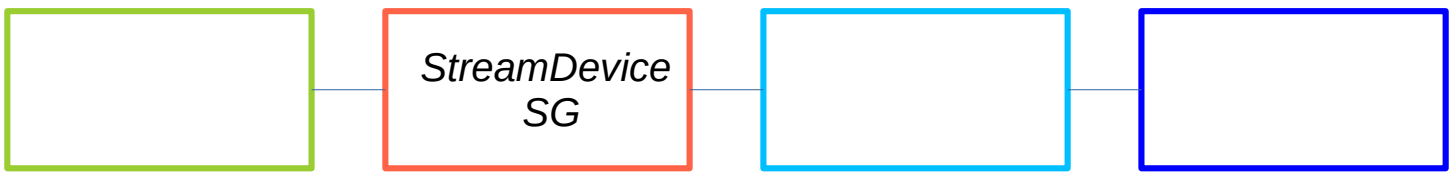
- AudioReach comes with *AudioReachCreator*, a graphical editor to create and edit graphs and including a library of modules
- To build a graph, first modules are grouped into **sub-graphs** (SGs)
- Then each sub-graph is placed in one of four possible slots within the graph
- Slots are chosen on the basis of what the sub-graphs' modules do:




*Modules that send samples to audio interface*

# Graph Design (Simplified)

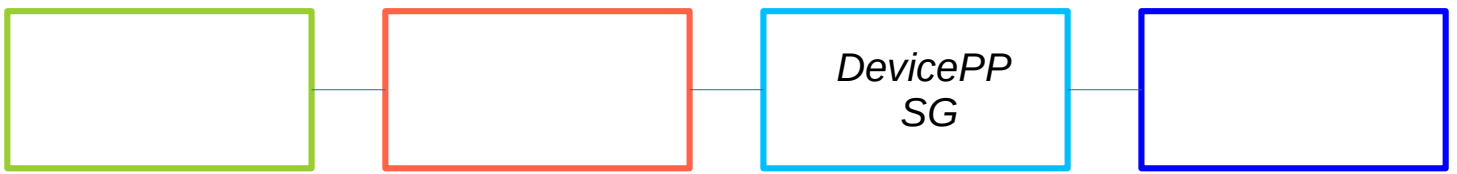
- AudioReach comes with *AudioReachCreator*, a graphical editor to create and edit graphs and including a library of modules
- To build a graph, first modules are grouped into **sub-graphs** (SGs)
- Then each sub-graph is placed in one of four possible slots within the graph
- Slots are chosen on the basis of what the sub-graphs' modules do:




*Modules that do format and sample rate conversion*

# Graph Design (Simplified)

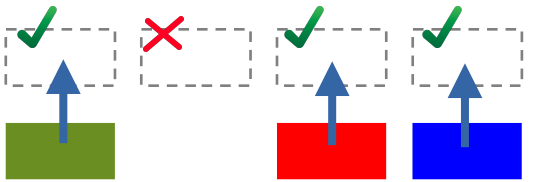
- AudioReach comes with *AudioReachCreator*, a graphical editor to create and edit graphs and including a library of modules
- To build a graph, first modules are grouped into **sub-graphs** (SGs)
- Then each sub-graph is placed in one of four possible slots within the graph
- Slots are chosen on the basis of what the sub-graphs' modules do:



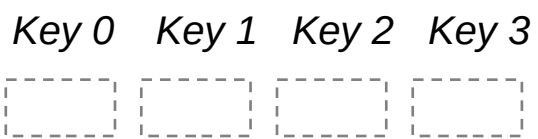

*Modules that carry out any processing before output to device*

# Graph Loading (Simplified)

- Once the graph is built, it's identified by:
  - the list of slots that are filled (out of four)
  - what fills them (sub-graphs and their modules)



- AudioReach uses a **key–value system** to manage the identification process:
  - Each slot is associated with a fixed **key**
  - Each sub-graph is associated with a chosen numerical **value**

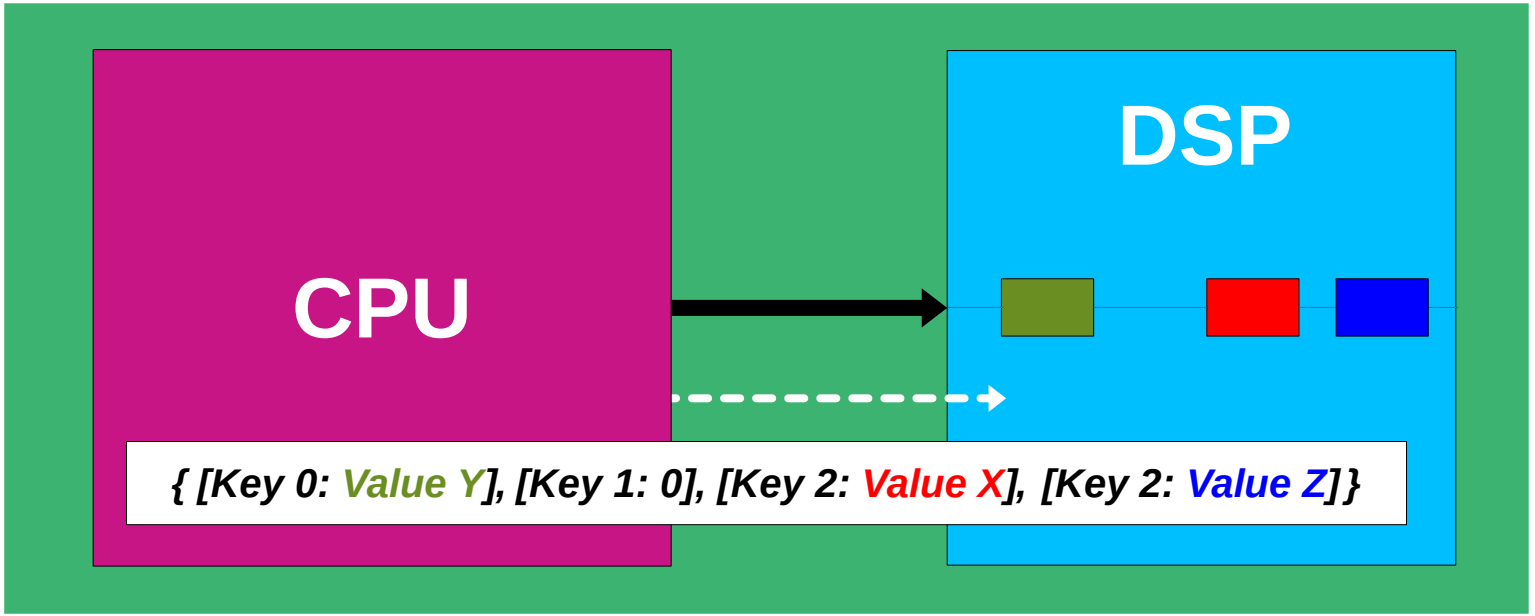




*Value 0 means ‘no sub-graph/empty slot’*

# Graph Key Vector

- CPU applications must assemble **a list of key–value** pairs to describe the graph that needs to be loaded on the DSP
- This list, called the *Graph Key Vector*, is sen*t to AudioReach at run time to identify the graph and load it!



# Audio Engine Default Graph

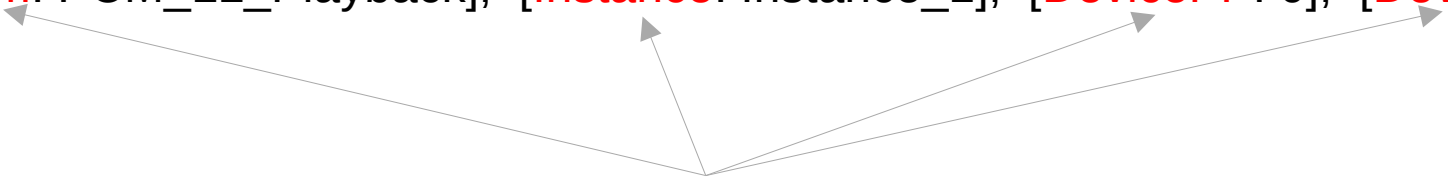
- The audio engine we are using sends via the AudioReach API the following default Graph Key Vector:

{ [Stream: PCM_LL_Playback], [Instance: Instance_1], [DevicePP: 0], [Device: Speaker] }

# Audio Engine Default Graph

- The audio engine we are using sends via the AudioReach API the following default Graph Key Vector:

{ [Stream: PCM_LL_Playback], [Instance: Instance_1], [DevicePP: 0], [Device: Speaker] }

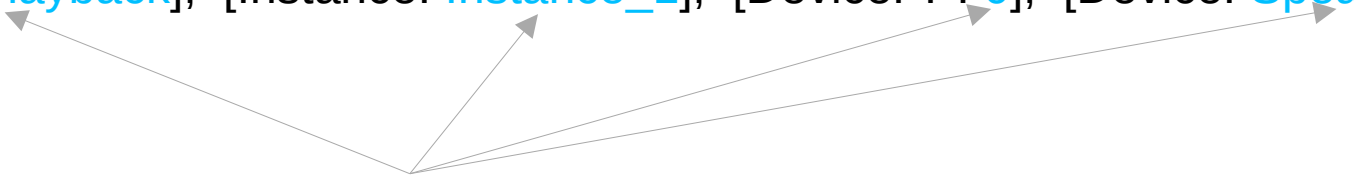


Fixed keys

# Audio Engine Default Graph

- The audio engine we are using sends via the AudioReach API the following default Graph Key Vector:

{ [Stream: PCM_LL_Playback], [Instance: Instance_1], [DevicePP: 0], [Device: Speaker] }



Values (as macros)

# Audio Engine Default Graph

- The audio engine we are using sends via the AudioReach API the following default Graph Key Vector:

{ [Stream: PCM_LL_Playback], [Instance: Instance_1], [DevicePP: 0], [Device: Speaker] }

- This means that all projects will load the same default graph on the DSP
- Let’s open AudioReachCreator and have a look at this graph...

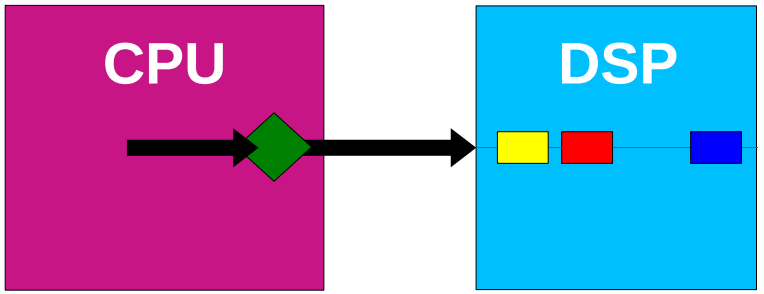
# 2. DSP Offloading

# Why DSP?

- DSPs are processors designed to process streams of digital signals
- Most common operations in digital signal processing: filters, convolution, mixing, transforms
  - All composed of sums of products
  - All working on buffers
- Hence DSPs:
  - support **MAC** (multiply–accumulate) as a single instruction and are equipped with a few MAC units working in **parallel**
  - handle **circular buffers in hardware**—no index arithmetic needed
  - are designed for **fixed-point** arithmetic (integer math) with **saturation** (overflow clips instead of wrapping!)

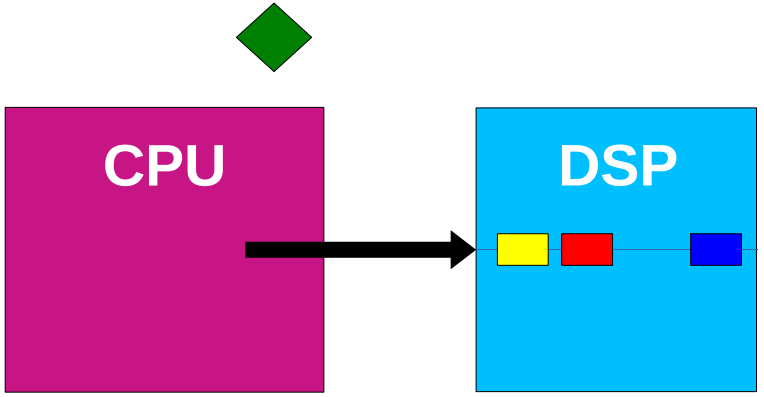
# Offloading on Snapdragon

- Let’s assume we designed a CPU audio application that runs some processing on the incoming samples (e.g., an audio effect)



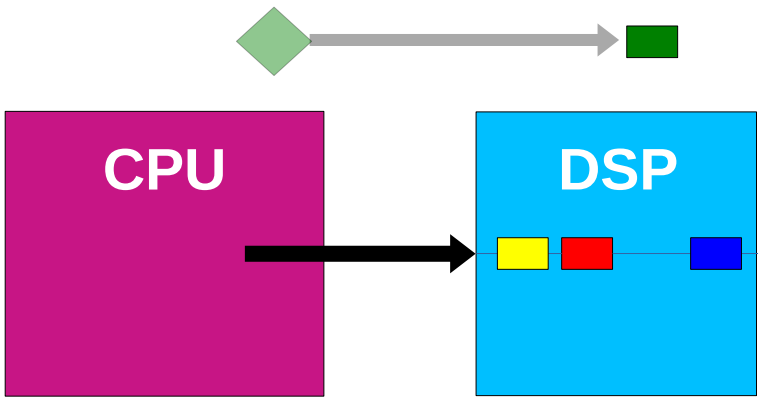
# Offloading on Snapdragon

- Let’s assume we designed a CPU audio application that runs some processing on the incoming samples (e.g., an audio effect)
- To offload that processing to the DSP, we remove it from the CPU code…



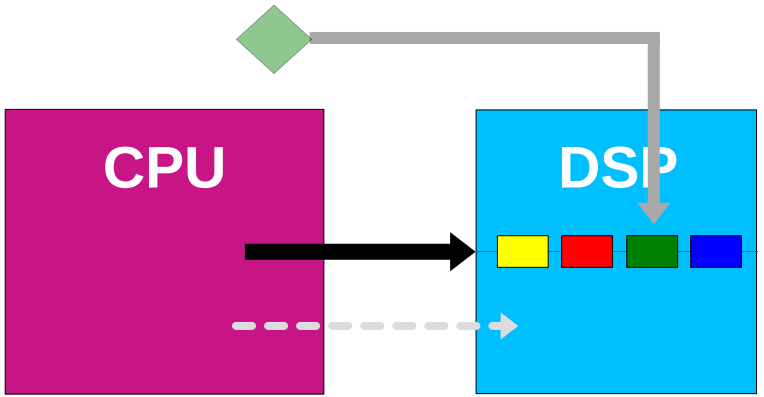
# Offloading on Snapdragon

- Let’s assume we designed a CPU audio application that runs some processing on the incoming samples (e.g., an audio effect)
- To offload that processing to the DSP, we remove it from the CPU code…
- ...and re-implement the same algorithm as a dedicated sub-graph, composed of one or more modules



# Offloading on Snapdragon

- Let’s assume we designed a CPU audio application that runs some processing on the incoming samples (e.g., an audio effect)
- To offload that processing to the DSP, we remove it from the CPU code…
- ...and re-implement the same algorithm as a dedicated sub-graph, composed of one or more modules
- Finally we create a new graph with it and load it at run time!



# Offloading Example

- Let’s have a look at the project ***projects/file_player_reverb***…

# Free Lunch?

- Offloading relieved the CPU load… but is there any **hidden cost**?

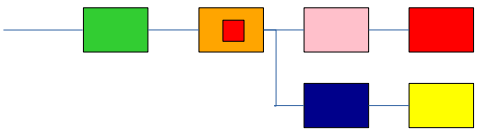
  Yes /:
- When we add a new sub-graph, we increase the **latency** of the overall DSP chain!
  - A new sub-graph means a longer graph with more modules
  - Each module carries out some processing… and it needs time to do that!
  - AudioReach allocates a new **buffer** for the sub-graph, to give all its modules time to process the samples within real-time deadlines
- This is a real constraint of DSP offloading!
- Our new sub-graph is set to the lowest latency setting, i.e., 1 ms

# Improving Performance

- Luckily, there are two ways around this added latency!
  - Graph design (AudioReachCreator)
  - Real-time audio parameters (CPU/run time)

# Improving Performance – Graph Design

- New modules can be added inside *existing* sub-graphs
  - Same graph length often means same latency!
- Some rules apply though:
  - The DSP may not run fast enough to run all modules in real time (chocking the DSP to relieve the CPU…)
  - Some specialized modules may still request additional computational time, increasing overall latency even if the number of sub-graphs remains the same
  - Sub-gaphs can be shared across graphs! Editing one with a new module may impact other graphs that are designed for different use cases



*AudioReachCreator is a sophisticated tool—graph design takes practice*

# Improving Performance – Audio Params

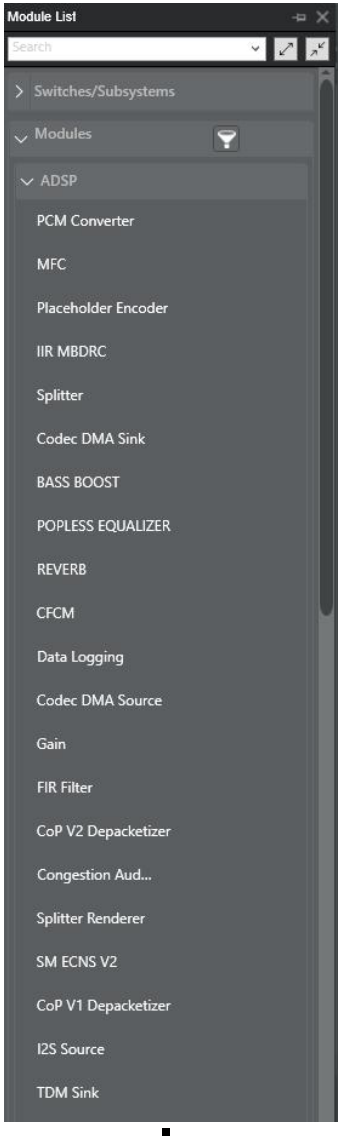
- When latency increases on the DSP side, it may be possible to reduce it on the CPU side!
- Offloading creates CPU overhead
- This may allow to run the CPU application with a much smaller audio buffer
  - Smaller audio buffer = lower latency!

# Improving Our App’s Performance

- The CPU app could run with a *period size* of no less than 96 frames
  - 96 frames @ 48 kHz → **4 ms CPU playback latency**
- With a ~9% CPU/core usage, our offloaded application’s period size can be easily set to the minimum value supported by the hardware, i.e., 48 frames
  - Let’s test it...
- Jumping from 96 to 48 frames means cutting down CPU latency in half!
  - **From 4 ms to 2 ms!**
- We not only canceled the **+1 ms** latency penalty of the DSP offload...
- ...but also improved the latency of the original application!
  - **Net −1 ms**

# Custom DSP Modules

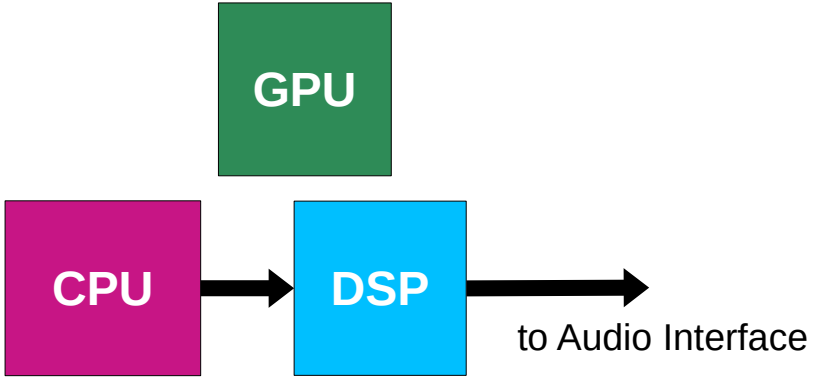
- AudioReachCreator includes a long list of modules that can be used and customized to design graphs
- But what if we want to offload an algorithm that is not implemented by any existing module?
  - You can build your own!
- AudioReach comes with *CAPI* (Common Audio Processor Interface), an API to write your own DSP modules in C



# 3. GPU Neural Audio Inference

# GPU and Audio

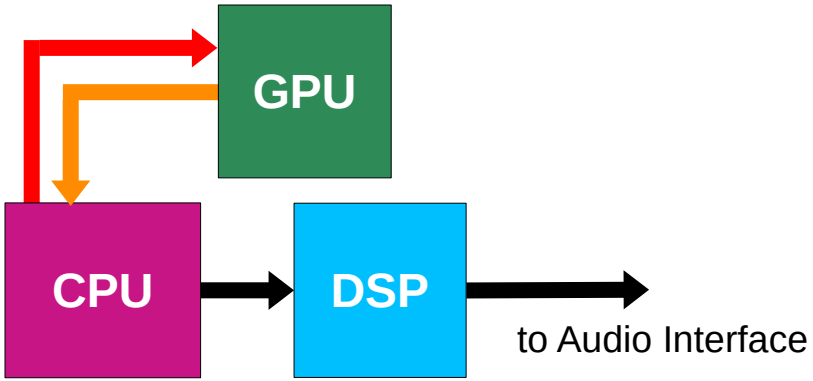
- On Snapdragon, audio always travels through the DSP to reach the audio interface, hence DSP offloading simply boils down to using a new/dedicated graph
- In contrast, the GPU is not a default part of the audio pipeline (on Snapdragon and anywhere else)!
  - Designed to process graphics
  - But flexible enough to carry out other tasks (‘GPGPU’), including audio



# GPU Audio Pipeline

- To use the GPU as an audio accelerator, we need to:
  - bring the audio data to the GPU, where it's processed
  - then bring the result back to the CPU
- From the CPU processed samples are again streamed into the standard DSP → interface pipeline

*Issue 1: moving data can be costly!*



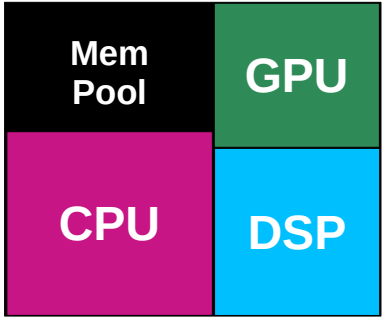
# GPU Parallel Processing

- GPUs pack *thousands* of small cores (arithmetic and logic units), all running the same operation on different data at once
- Hence they require **kernels**, i.e., programs that execute simultaneously across as many cores as possible
  - Not a loop walking through data!
- If we want to run audio processing on a GPU, we need to implement our algorithm as a kernel

*Issue 2: writing parallelizable kernels is not simple—especially neural models!*

# Shared Memory

- On Snapdragon, the Adreno GPU shares a single memory pool with the rest of the silicon
  - Unified memory architecture
- This means that moving data to and from the GPU is less expensive than in other systems:
  - discrete GPUs (transfer across a bus to dedicated VRAM)
  - most integrated GPUs (same DRAM, but still a copy between separate address spaces)




SoC

*Issue 1:*
*largely solved!*

# Qualcomm AI Runtime SDK

- Then, Qualcomm provides QAIRT (Qualcomm AI Runtime), an SDK for deploying neural models on Qualcomm hardware, including Snapdragon
- It contains:
  - tools to convert trained models into **optimized on-device representations** that are backend-independent (*backend* = target processor where the model will run)
  - **QNN**, a low-level API and the run-time libraries that load a converted model and execute it on a **chosen backend**
    - Including the GPU!

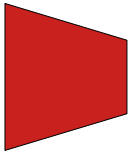
*Issue 2:*
*largely solved too!*

# QAIRT Pipeline

- We start from a model pre-trained in one of the main frameworks

PyTorch
TensorFlow
ONNX

*Model*

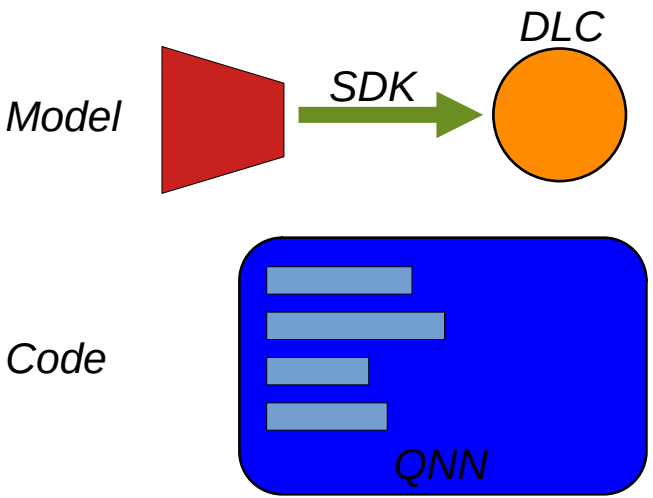


# QAIRT Pipeline

- We start from a model pre-trained in one of the main frameworks
- Then with the SDK conversion tool we generate a DLC (Deep Learning Container)
  - i.e., the backend-independent format that is consumed at run time

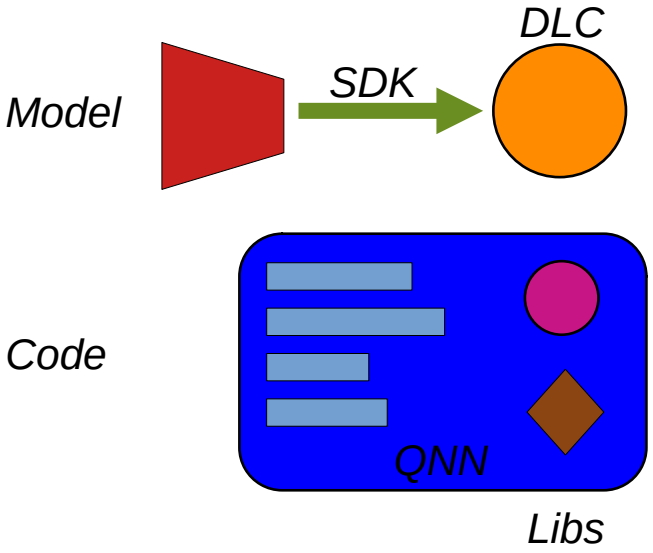
# QAIRT Pipeline

- Our C++ code has to load two dynamic libs, accessible via the QNN API:



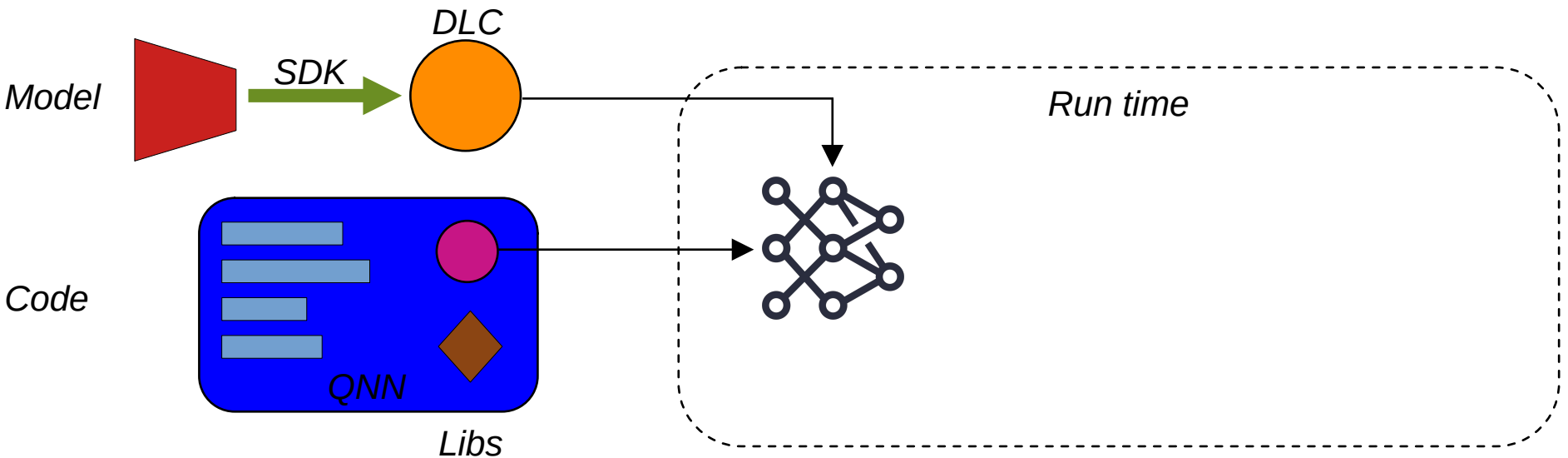
# QAIRT Pipeline

- Our C++ code has to load two dynamic libs, accessible via the QNN API:
  - the system library (*libQnnSystem.so*)
  - a backend library (e.g., *libQnnCpu.so*, *libQnnGpu.so*)



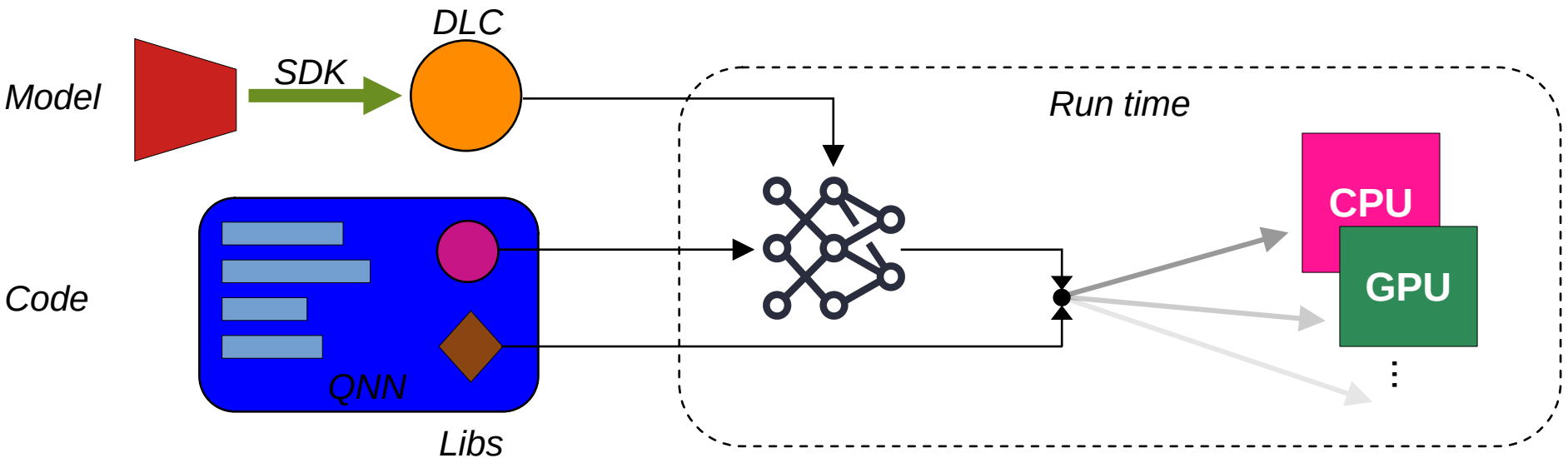
# QAIRT Pipeline

- At run-time, the libs carry out different roles:
  - the system library parses the DLC and composes the model's **graph**



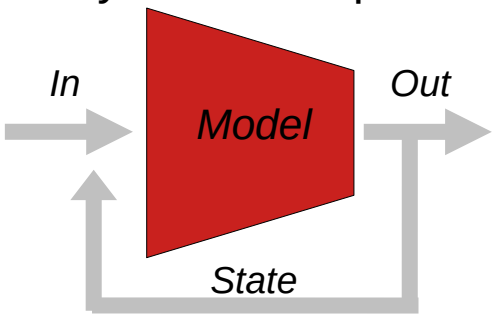
# QAIRT Pipeline

- At run-time, the libs carry out different roles:
  - the system library parses the DLC and composes the model's graph
  - the chosen backend library finalizes that graph for its target backend, allocates the tensors and executes it



# Neural Kernels

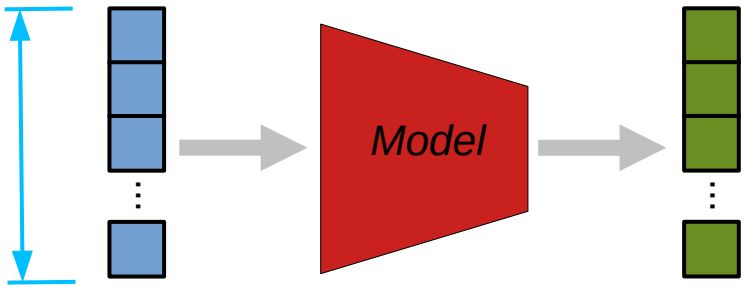
- When the target is the GPU, the backend maps each layer of the graph onto a pre-written parallel kernel
  - Kernels are designed in OpenCL and tuned for Adreno!
- This mapping is automatic (via QNN) but has a **caveat**...
  - All QNN graphs are **stateless**, i.e., nothing persists between inference calls
- This means that *stateful* Python models (e.g., those with hidden states, streaming caches) need to be modified **before conversion**
  - The model must pass any state as input and return it as output at every call




*State fed back!*

# Neural Audio Kernels

- When running neural audio models, we have one inference call per audio callback (*render()* function)
- The model receives and processes a **buffer of samples** (neural processing) OR synthesizes a buffer of samples from scratch (neural synthesis)
- Each layer's kernel runs across all the samples simultaneously*
- Each inference returns a new buffer of samples
  - The buffer size determines how much parallelism the GPU gets!



# Neural Audio Inference Example

- Let’s have a look at the project ***projects/brave***…

# Some Notes on GPU Acceleration

*“Each layer's kernel runs across all the samples simultaneously*”*

- *Actually, not all layers can parallelize across time!
  - For example, in recurrent layers sample *n+1* depends on sample *n*, so the kernel has to run through the buffer sequentially, destroying the GPU speed-up
- In those that can run in parallel, the speed-up depends on the buffer size, but the buffer size also heavily impacts the overall latency of the application!
  - Small models may start showing meaningful speed-ups at buffer sizes that are already too large for interactive audio
- In the above cases, it may be better to stay on the CPU or to choose the DSP as the target backend
  - QNN has a dedicated libs for the Hexagon too, though models need to be quantized to *int16* or *int8*

# Offloading $\stackrel{?}{=}$ Acceleration

- In our application, inference got offloaded to the GPU where it was accelerated
- Yet, in general **offloading and acceleration are not the same concept**
- In our reverb example, the algorithm was offloaded to the DSP, but we cannot claim it was accelerated
  - DSP is probably not faster here: 4 MACs/cycle at a low clock, against a 1.9 GHz Cortex-A55 with NEON
  - The win was CPU headroom, power and latency—not necessarily speed
- That said, the DSP can be a real accelerator!
  - Via optimized implementations (CAPI is an approach)